Data Eng

In [1]:
```
import findspark
findspark.init()

from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import *
import pandas as pd


spark = SparkSession.builder \
    .appName("DE_DemandForecasting") \
    .master("local[*]") \
    .config("spark.sql.adaptive.enabled", "true") \
    .config("spark.sql.adaptive.coalescePartitions.enabled", "true") \
    .getOrCreate()

print("Spark Version:", spark.version)
```

```
26/05/02 00:43:16 WARN Utils: Your hostname, DESKTOP-8HUS7BO resolves to a loop
back address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
26/05/02 00:43:16 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLev
el(newLevel).
26/05/02 00:43:17 WARN NativeCodeLoader: Unable to load native-hadoop library f
or your platform... using builtin-java classes where applicable
Spark Version: 3.5.2
```

In [2]:
```
pdf = pd.read_excel("data.xlsx")

print(f"Raw data shape: {pdf.shape}")
```

```
Raw data shape: (541909, 8)
```

In [3]:
```
schema = StructType([
    StructField("InvoiceNo",   StringType(),  True),
    StructField("StockCode",   StringType(),  True),
    StructField("Description", StringType(),  True),
    StructField("Quantity",    IntegerType(), True),
    StructField("InvoiceDate", StringType(),  True),
    StructField("UnitPrice",   DoubleType(),  True),
    StructField("CustomerID",  DoubleType(),  True),
    StructField("Country",     StringType(),  True),
])

pdf["Quantity"]    = pd.to_numeric(pdf["Quantity"],   errors="coerce")
pdf["UnitPrice"]   = pd.to_numeric(pdf["UnitPrice"],  errors="coerce")
pdf["CustomerID"]  = pd.to_numeric(pdf["CustomerID"], errors="coerce")
pdf["InvoiceDate"] = pdf["InvoiceDate"].astype(str)
```

```
df = spark.createDataFrame(pdf, schema=schema)
print(f"Total rows: {df.count()}")
```

```
26/05/02 00:44:25 WARN TaskSetManager: Stage 0 contains a task of very large si
ze (5154 KiB). The maximum recommended task size is 1000 KiB.
[Stage 0:>                                                          (0 + 8) /
8]
Total rows: 541909
```

In [4]:
```
# Drop no product description
df_clean = df.dropna(subset=["Description"])

# flag cancelled orders
df_clean = df_clean.withColumn("IsCancelled", F.col("InvoiceNo").startswith("C

# split valid and cancelled
df_cancelled = df_clean.filter(F.col("IsCancelled") == True)
df_valid     = df_clean.filter(F.col("IsCancelled") == False)

# valid only
df_valid = df_valid.filter((F.col("Quantity") > 0) & (F.col("UnitPrice") > 0))

# Cast CustomerID to Integer
df_valid = df_valid.withColumn("CustomerID", F.col("CustomerID").cast(IntegerT

# Revenue
df_valid = df_valid.withColumn("Revenue", F.round(F.col("Quantity") * F.col("U
```

In [5]:
```
# Parse dates
df_valid = df_valid.withColumn("InvoiceDate", F.to_timestamp("InvoiceDate"))
df_valid = df_valid.filter(F.col("InvoiceDate").isNotNull())
# Extract temporal features
df_valid = (df_valid
    .withColumn("Year",       F.year("InvoiceDate"))
    .withColumn("Month",      F.month("InvoiceDate"))
    .withColumn("Day",        F.dayofmonth("InvoiceDate"))
    .withColumn("DayOfWeek",  F.dayofweek("InvoiceDate"))   # 1=Sun, 7=Sat
    .withColumn("WeekOfYear", F.weekofyear("InvoiceDate"))
    .withColumn("Quarter",    F.quarter("InvoiceDate"))
    .withColumn("Hour",       F.hour("InvoiceDate"))
    .withColumn("YearMonth",  F.date_format("InvoiceDate", "yyyy-MM"))
)
```

In [6]:
```
# Weekend flag
df_valid = df_valid.withColumn(
    "IsWeekend",
    F.when(F.col("DayOfWeek").isin([1, 7]), 1).otherwise(0)
)

print(f"Valid orders:     {df_valid.count()}")
print(f"Cancelled orders: {df_cancelled.count()}")
```

```
26/05/02 00:45:01 WARN TaskSetManager: Stage 3 contains a task of very large si
ze (5154 KiB). The maximum recommended task size is 1000 KiB.

Valid orders:     530104
26/05/02 00:45:03 WARN TaskSetManager: Stage 6 contains a task of very large si
ze (5154 KiB). The maximum recommended task size is 1000 KiB.
[Stage 6:>                                                          (0 + 8) /
8]
Cancelled orders: 9288
```

In [ ]:
```
# write to HDFS
HDFS_CLEANED   = "hdfs://localhost:9000/data/retail/cleaned"
HDFS_CANCELLED = "hdfs://localhost:9000/data/retail/cancelled"

df_valid.write.mode("overwrite").partitionBy("YearMonth").parquet(HDFS_CLEANED
df_cancelled.write.mode("overwrite").parquet(HDFS_CANCELLED)

print("saved to HDFS")


import subprocess
subprocess.run(["hdfs", "dfs", "-setrep", "3", HDFS_CLEANED], capture_output=T
subprocess.run(["hdfs", "dfs", "-setrep", "3", HDFS_CANCELLED], capture_output
print("Replication factor set to 3")

# Verify
print("\n--- HDFS Listing ---")
subprocess.run(["hdfs", "dfs", "-ls", "-R", HDFS_CLEANED])

print("Done")
spark.stop()
```

Data Analyst

In [1]:
```
import findspark
findspark.init()

from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import *

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
plt.style.use("seaborn-v0_8-whitegrid")
sns.set_palette("husl")

spark = SparkSession.builder \
```

```
    .appName("DE_DemandForecasting") \
    .master("local[*]") \
    .config("spark.sql.adaptive.enabled", "true") \
    .config("spark.sql.adaptive.coalescePartitions.enabled", "true") \
    .getOrCreate()
print("spark version : ", spark.version)
```

```
/home/hadoop/.local/lib/python3.10/site-packages/matplotlib/projections/__ini
t__.py:63: UserWarning: Unable to import Axes3D. This may be due to multiple ve
rsions of Matplotlib being installed (e.g. as a system package and as a pip pac
kage). As a result, the 3D projection is not available.
  warnings.warn("Unable to import Axes3D. This may be due to multiple versions
of "
26/05/02 01:49:25 WARN Utils: Your hostname, DESKTOP-8HUS7BO resolves to a loop
back address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
26/05/02 01:49:25 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLev
el(newLevel).
26/05/02 01:49:26 WARN NativeCodeLoader: Unable to load native-hadoop library f
or your platform... using builtin-java classes where applicable
spark version :  3.5.2
```

In [2]:
```
HDFS_CLEANED   = "hdfs://localhost:9000/data/retail/cleaned"
HDFS_CANCELLED = "hdfs://localhost:9000/data/retail/cancelled"
HDFS_OUTPUT    = "hdfs://localhost:9000/data/retail/outputs"


df = spark.read.parquet(HDFS_CLEANED)
df_cancelled = spark.read.parquet(HDFS_CANCELLED)

print("ok")
```

```
ok
```

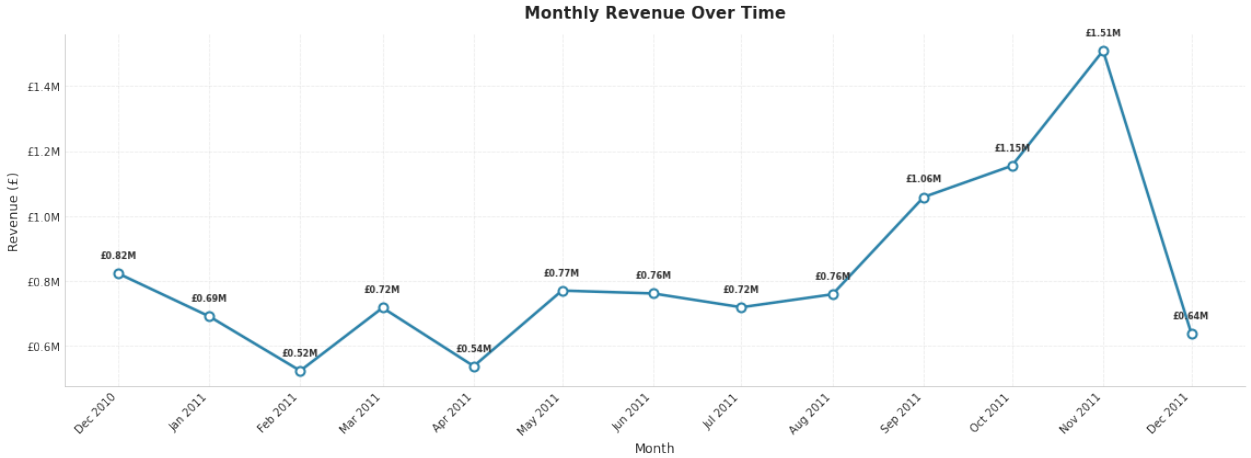
In [4]:
```
df.select("Quantity", "UnitPrice", "Revenue").describe().show()

total_revenue   = df.agg(F.sum("Revenue")).collect()[0][0]
total_orders    = df.select("InvoiceNo").distinct().count()
total_customers = df.select("CustomerID").distinct().count()
total_products  = df.select("StockCode").distinct().count()
avg_order_value = df.groupBy("InvoiceNo").agg(F.sum("Revenue").alias("OrderVal
avg_order_value = avg_order_value.agg(F.avg("OrderValue")).collect()[0][0]

print(f"Total Revenue     : {total_revenue:,.2f}")
print(f"Unique Orders     : {total_orders:,}")
print(f"Unique Customers  : {total_customers:,}")
print(f"Unique Products   : {total_products:,}")
print(f"Average Order Value: {avg_order_value:,.2f}")
```

```
+-------+------------------+------------------+------------------+
|summary|          Quantity|         UnitPrice|           Revenue|
+-------+------------------+------------------+------------------+
|  count|            530104|            530104|            530104|
|   mean|10.542037034242338| 3.907625247122595|20.121871444091383|
| stddev|155.52412351063685|35.915681104255526| 270.3567432189682|
|    min|                 1|             0.001|               0.0|
|    max|             80995|          13541.33|          168469.6|
+-------+------------------+------------------+------------------+

Total Revenue     : £10,666,684.54
Unique Orders     : 19,960
Unique Customers  : 4,339
Unique Products   : 3,922
Average Order Value: 534.40
```

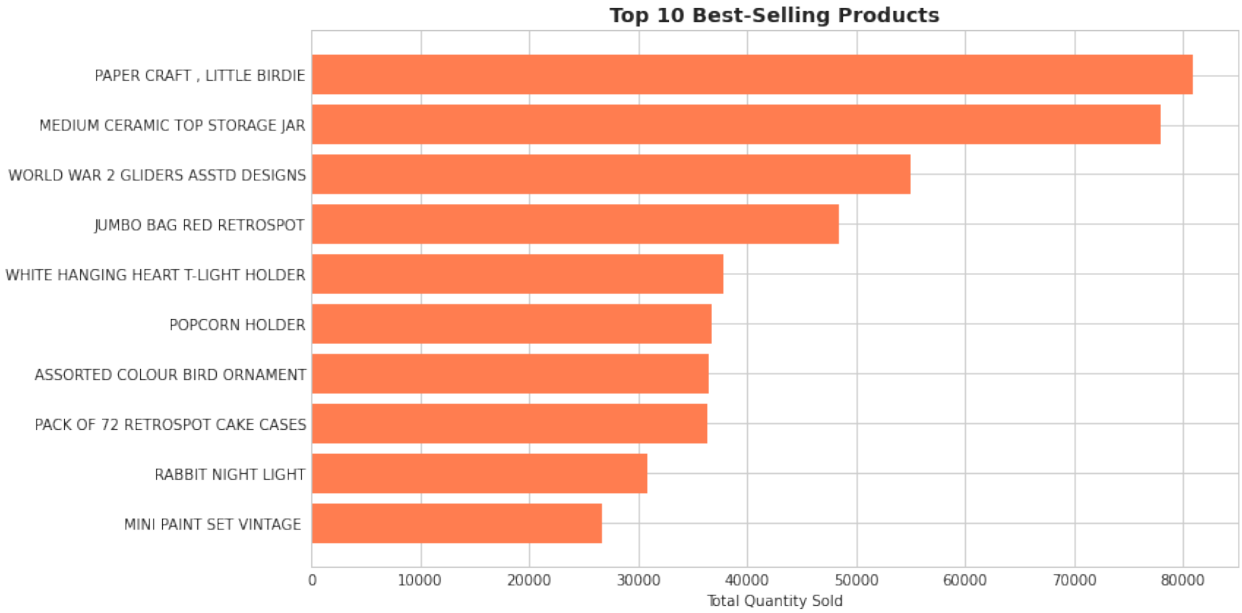
In [9]:

```python
import matplotlib.dates as mdates

monthly = (df.groupBy("Year", "Month")
            .agg(F.round(F.sum("Revenue"), 2).alias("TotalRevenue"))
            .orderBy("Year", "Month")).toPandas()

monthly["YearMonth"] = pd.to_datetime(
    monthly["Year"].astype(str) + "-" + monthly["Month"].astype(str) + "-01"
)

fig, ax = plt.subplots(figsize=(16, 6))

ax.plot(monthly["YearMonth"], monthly["TotalRevenue"],
        marker="o", color="#2E86AB", linewidth=2.5, markersize=8,
        markerfacecolor="white", markeredgewidth=2, markeredgecolor="#2E86AB")

# Format x-axis
ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=1))
plt.setp(ax.xaxis.get_majorticklabels(), rotation=45, ha="right")

for _, row in monthly.iterrows():
    val = row["TotalRevenue"]
    label = f"£{val/1e6:.2f}M"
    ax.text(row["YearMonth"], val + 40000, label,
            ha="center", va="bottom", fontsize=8, fontweight="bold", color="#3

ax.grid(True, alpha=0.3, linestyle="--")
ax.set_title("Monthly Revenue Over Time", fontsize=15, fontweight="bold", pad=
ax.set_xlabel("Month", fontsize=12)
ax.set_ylabel("Revenue (£)", fontsize=12)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f"£{x/1e6:.1f}M"))

# Cleaner look
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
```

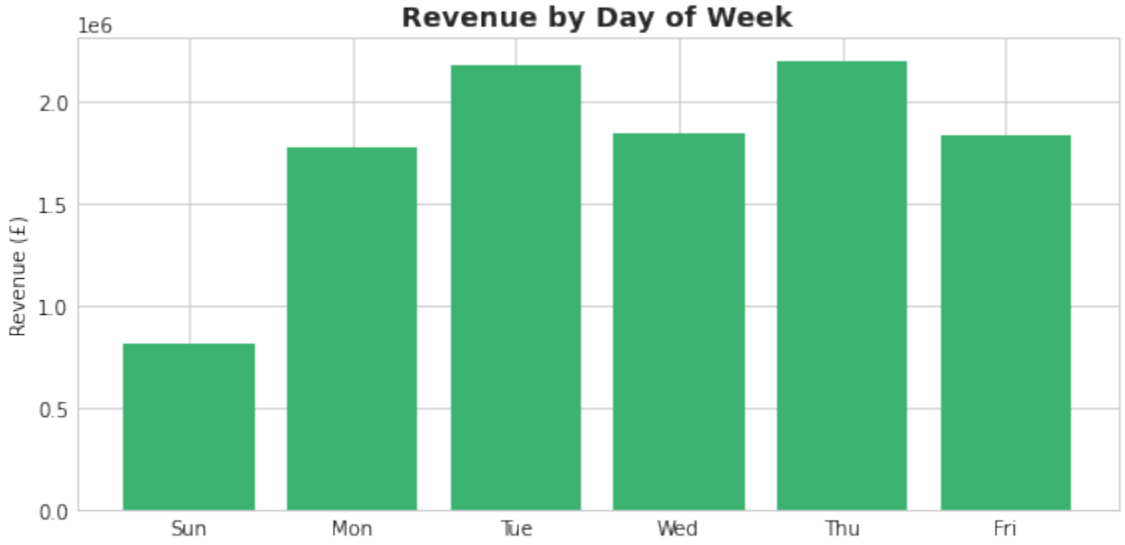
```
plt.tight_layout()
plt.savefig("/tmp/plot_01_monthly_revenue.png", dpi=150, bbox_inches="tight")
plt.show()
```



In [10]:
```
# top selling products
top_products = (df.groupBy("Description")
                  .agg(F.sum("Quantity").alias("TotalQty"))
                  .orderBy(F.desc("TotalQty"))
                  .limit(10))
tp_pd = top_products.toPandas()

plt.figure(figsize=(12, 6))
plt.barh(tp_pd["Description"][::-1], tp_pd["TotalQty"][::-1], color="coral")
plt.title("Top 10 Best-Selling Products", fontsize=14, fontweight="bold")
plt.xlabel("Total Quantity Sold")
plt.tight_layout()
plt.savefig("/tmp/plot_02_top_products.png", dpi=150)
plt.show()
```

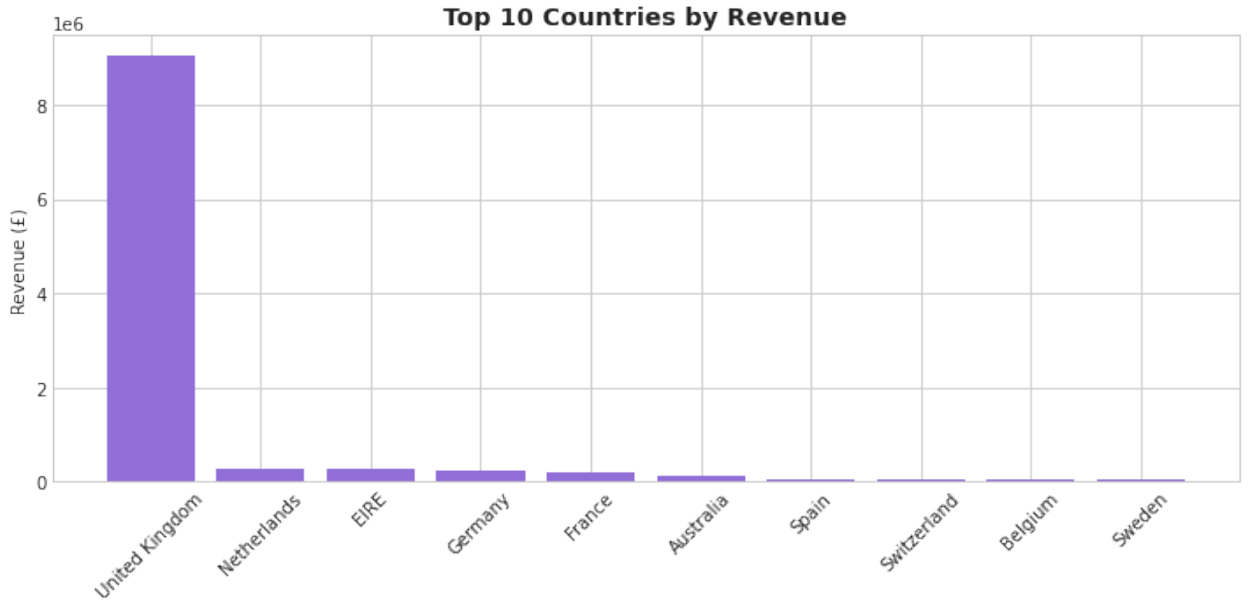


In [11]:
```
# by day of week
day_map = {1:"Sun", 2:"Mon", 3:"Tue", 4:"Wed", 5:"Thu", 6:"Fri", 7:"Sat"}
dow = (df.groupBy("DayOfWeek")
         .agg(F.sum("Revenue").alias("TotalRevenue"))
         .orderBy("DayOfWeek")).toPandas()
dow["DayName"] = dow["DayOfWeek"].map(day_map)

plt.figure(figsize=(8, 4))
plt.bar(dow["DayName"], dow["TotalRevenue"], color="mediumseagreen")
plt.title("Revenue by Day of Week", fontsize=14, fontweight="bold")
plt.ylabel("Revenue (£)")
plt.tight_layout()
plt.savefig("/tmp/plot_03_dow_revenue.png", dpi=150)
plt.show()
```



In [13]:
```
# by country
```

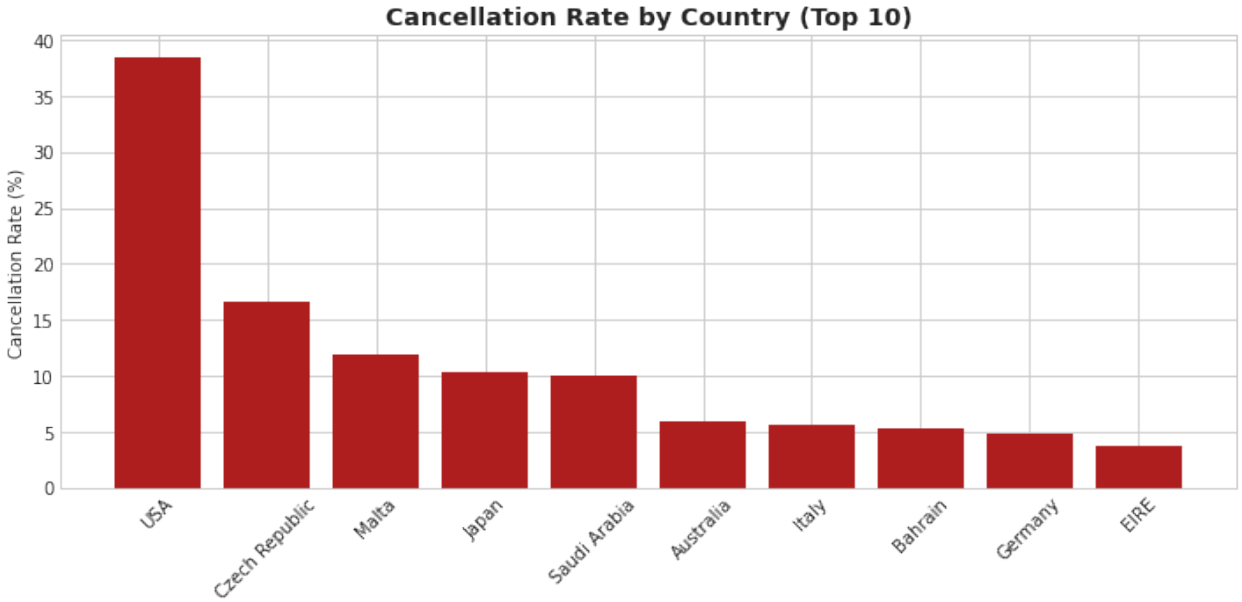
```
countries = (df.groupBy("Country")
               .agg(F.round(F.sum("Revenue"), 2).alias("TotalRevenue"))
               .orderBy(F.desc("TotalRevenue"))
               .limit(10)).toPandas()

plt.figure(figsize=(10, 5))
plt.bar(countries["Country"], countries["TotalRevenue"], color="mediumpurple")
plt.title("Top 10 Countries by Revenue", fontsize=14, fontweight="bold")
plt.ylabel("Revenue (£)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("/tmp/plot_04_countries.png", dpi=150)
plt.show()
```

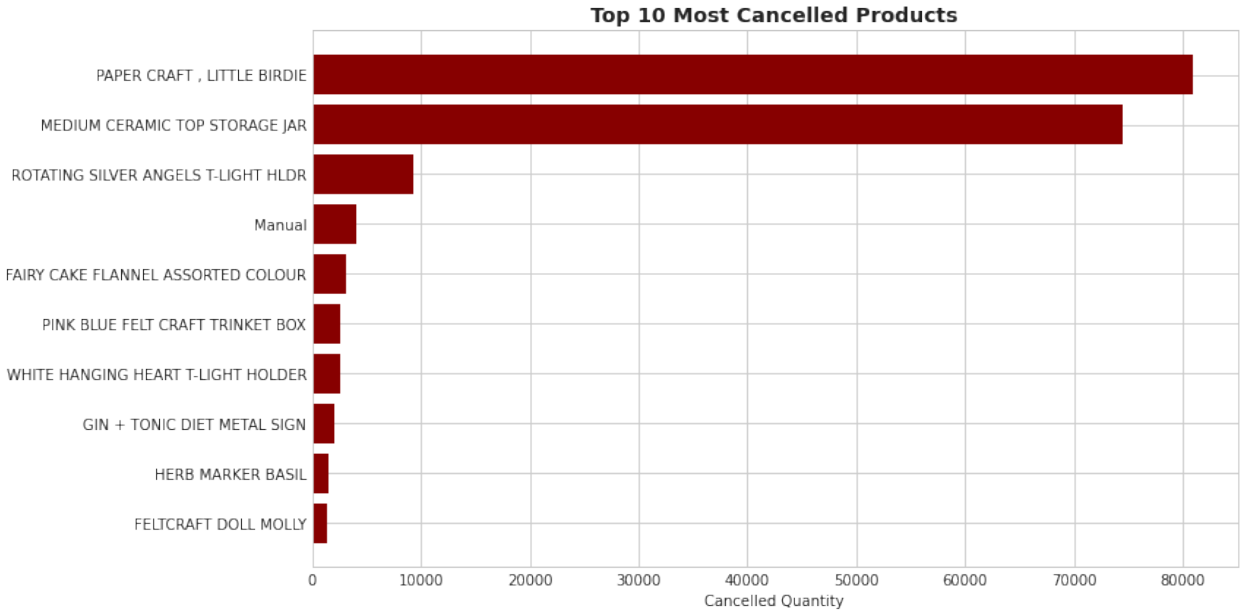


In [14]:
```
print("Cancelled Order Analysis")

# Cancellation rate overall
cancelled_count = df_cancelled.count()
valid_count     = df.count()
cancel_rate     = cancelled_count / (valid_count + cancelled_count) * 100
print(f"Cancellation Rate: {cancel_rate:.2f}%")

# Revenue lost to cancellations
cancelled_revenue = df_cancelled.withColumn(
    "LostRevenue", F.abs(F.col("Quantity") * F.col("UnitPrice"))
).agg(F.sum("LostRevenue")).collect()[0][0]
print(f"Estimated Revenue Lost to Cancellations: £{cancelled_revenue:,.2f}")

# Cancellation rate by country
cancel_by_country = (df_cancelled.groupBy("Country")
                     .agg(F.count("*").alias("CancelledCount")))
valid_by_country  = (df.groupBy("Country")
                     .agg(F.count("*").alias("ValidCount")))
```

```
country_rates = (valid_by_country
    .join(cancel_by_country, "Country", "left_outer")
    .fillna(0)
    .withColumn("CancelRate",
        F.round(F.col("CancelledCount") / (F.col("ValidCount") + F.col("Cancel
    .orderBy(F.desc("CancelRate"))
    .limit(10)).toPandas()

plt.figure(figsize=(10, 5))
plt.bar(country_rates["Country"], country_rates["CancelRate"], color="firebric
plt.title("Cancellation Rate by Country (Top 10)", fontsize=14, fontweight="bo
plt.ylabel("Cancellation Rate (%)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("/tmp/plot_05_cancel_by_country.png", dpi=150)
plt.show()

# Cancellation by product (top 10 most cancelled)
cancel_by_product = (df_cancelled.groupBy("Description")
                     .agg(F.sum(F.abs("Quantity")).alias("CancelledQty"))
                     .orderBy(F.desc("CancelledQty"))
                     .limit(10)).toPandas()

plt.figure(figsize=(12, 6))
plt.barh(cancel_by_product["Description"][::-1], cancel_by_product["CancelledQ
plt.title("Top 10 Most Cancelled Products", fontsize=14, fontweight="bold")
plt.xlabel("Cancelled Quantity")
plt.tight_layout()
plt.savefig("/tmp/plot_06_cancel_by_product.png", dpi=150)
plt.show()
```

```
Cancelled Order Analysis
Cancellation Rate: 1.72%
Estimated Revenue Lost to Cancellations: £896,812.49
```





In [19]:

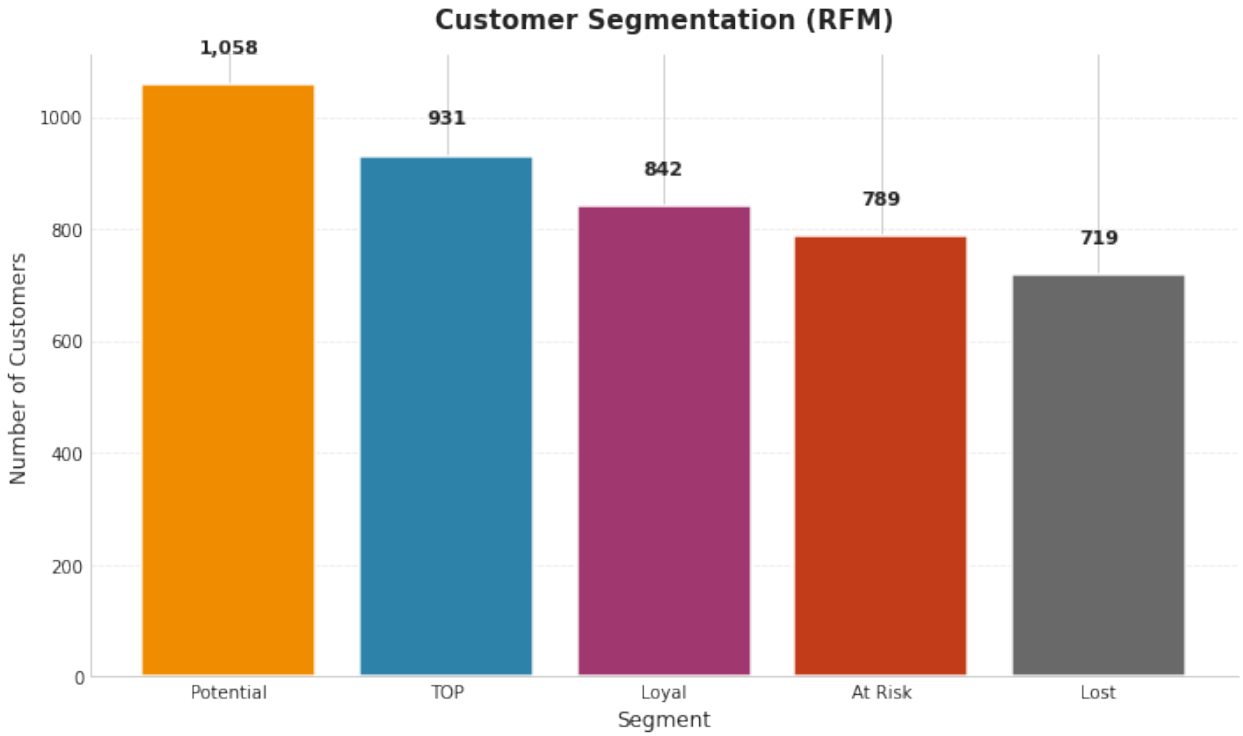
```python
from datetime import timedelta
from pyspark.sql.window import Window
from pyspark.sql.types import StringType

print("RFM Analysis")

max_date = df.agg(F.max("InvoiceDate")).collect()[0][0]
ref_date = max_date + timedelta(days=1)

rfm = (df.groupBy("CustomerID")
        .agg(
            F.datediff(F.lit(ref_date), F.max("InvoiceDate")).alias("Recency"),
            F.countDistinct("InvoiceNo").alias("Frequency"),
            F.round(F.sum("Revenue"), 2).alias("Monetary")
        ))

# Compute global quantile boundaries
quantiles_r = rfm.approxQuantile("Recency",   [0.2, 0.4, 0.6, 0.8], 0.01)
quantiles_f = rfm.approxQuantile("Frequency", [0.2, 0.4, 0.6, 0.8], 0.01)
quantiles_m = rfm.approxQuantile("Monetary",  [0.2, 0.4, 0.6, 0.8], 0.01)

# Recency: lower = more recent = better → score 5 for lowest values
@F.udf("int")
def r_score(val):
    if val is None:
        return None
    for i, b in enumerate(sorted(quantiles_r)):
        if val <= b:
            return 5 - i  # invert: lowest recency days → score 5
    return 1

# Frequency & Monetary: higher = better → score 5 for highest values
@F.udf("int")
def f_score(val):
```

```
    if val is None:
        return None
    for i, b in enumerate(sorted(quantiles_f)):
        if val <= b:
            return i + 1
    return 5

@F.udf("int")
def m_score(val):
    if val is None:
        return None
    for i, b in enumerate(sorted(quantiles_m)):
        if val <= b:
            return i + 1
    return 5

rfm = (rfm
    .withColumn("R_Score", r_score(F.col("Recency")))
    .withColumn("F_Score", f_score(F.col("Frequency")))
    .withColumn("M_Score", m_score(F.col("Monetary")))
)

# Segment scoring (max = 15, min = 3)
@F.udf(StringType())
def segment(r, f, m):
    score = r + f + m
    if score >= 13:
        return "TOP"
    elif score >= 10:
        return "Loyal"
    elif score >= 7:
        return "Potential"
    elif score >= 5:
        return "At Risk"
    else:
        return "Lost"

rfm = rfm.withColumn("Segment", segment(F.col("R_Score"), F.col("F_Score"), F.

rfm.select("CustomerID", "Recency", "Frequency", "Monetary",
           "R_Score", "F_Score", "M_Score", "Segment").show(10, truncate=False

# Distribution
segment_dist = (rfm.groupBy("Segment")
                .agg(
                    F.count("*").alias("Count"),
                    F.round(F.avg("Recency"), 1).alias("AvgRecency"),
                    F.round(F.avg("Frequency"), 1).alias("AvgFrequency"),
                    F.round(F.avg("Monetary"), 2).alias("AvgMonetary")
                )
                .orderBy(F.desc("Count"))).toPandas()

print("Segment Distribution:")
```

```
print(segment_dist.to_string(index=False))

# Plot
fig, ax = plt.subplots(figsize=(10, 6))
colors = {"TOP": "#2E86AB", "Loyal": "#A23B72", "Potential": "#F18F01",
          "At Risk": "#C73E1D", "Lost": "#6B6B6B"}
bar_colors = [colors.get(s, "gray") for s in segment_dist["Segment"]]
bars = ax.bar(segment_dist["Segment"], segment_dist["Count"], color=bar_colors

for bar in bars:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height + 50,
            f'{int(height):,}', ha='center', va='bottom', fontsize=11, fontwei

ax.set_title("Customer Segmentation (RFM)", fontsize=15, fontweight="bold", pa
ax.set_ylabel("Number of Customers", fontsize=12)
ax.set_xlabel("Segment", fontsize=12)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.grid(axis="y", alpha=0.3, linestyle="--")
plt.tight_layout()
plt.savefig("/tmp/plot_rfm_segments.png", dpi=150, bbox_inches="tight")
plt.show()

rfm.write.mode("overwrite").parquet(f"{HDFS_OUTPUT}/rfm_segments")
print("RFM saved to HDFS")
```

```
RFM Analysis
+----------+-------+---------+--------+-------+-------+-------+---------+
|CustomerID|Recency|Frequency|Monetary|R_Score|F_Score|M_Score|Segment  |
+----------+-------+---------+--------+-------+-------+-------+---------+
|15619     |11     |1        |336.4   |5      |1      |2      |Potential|
|15727     |17     |7        |5178.96 |4      |5      |5      |TOP      |
|15790     |11     |1        |220.85  |5      |1      |1      |Potential|
|13832     |20     |1        |52.2    |4      |1      |1      |At Risk  |
|13623     |31     |5        |747.78  |4      |4      |3      |Loyal    |
|15957     |32     |1        |428.89  |4      |1      |2      |Potential|
|16503     |107    |4        |1431.93 |2      |4      |4      |Loyal    |
|17389     |1      |34       |31833.68|5      |5      |5      |TOP      |
|17420     |51     |3        |598.83  |3      |3      |3      |Potential|
|16386     |29     |2        |317.2   |4      |2      |2      |Potential|
+----------+-------+---------+--------+-------+-------+-------+---------+
only showing top 10 rows
```

```
Segment Distribution:
  Segment  Count  AvgRecency  AvgFrequency  AvgMonetary
Potential   1058        75.0           2.2       893.36
      TOP    931        14.9          13.3      8397.50
    Loyal    842        44.1           4.1      1728.79
  At Risk    789       122.9           1.3       381.17
     Lost    719       245.3           1.0       204.53
```

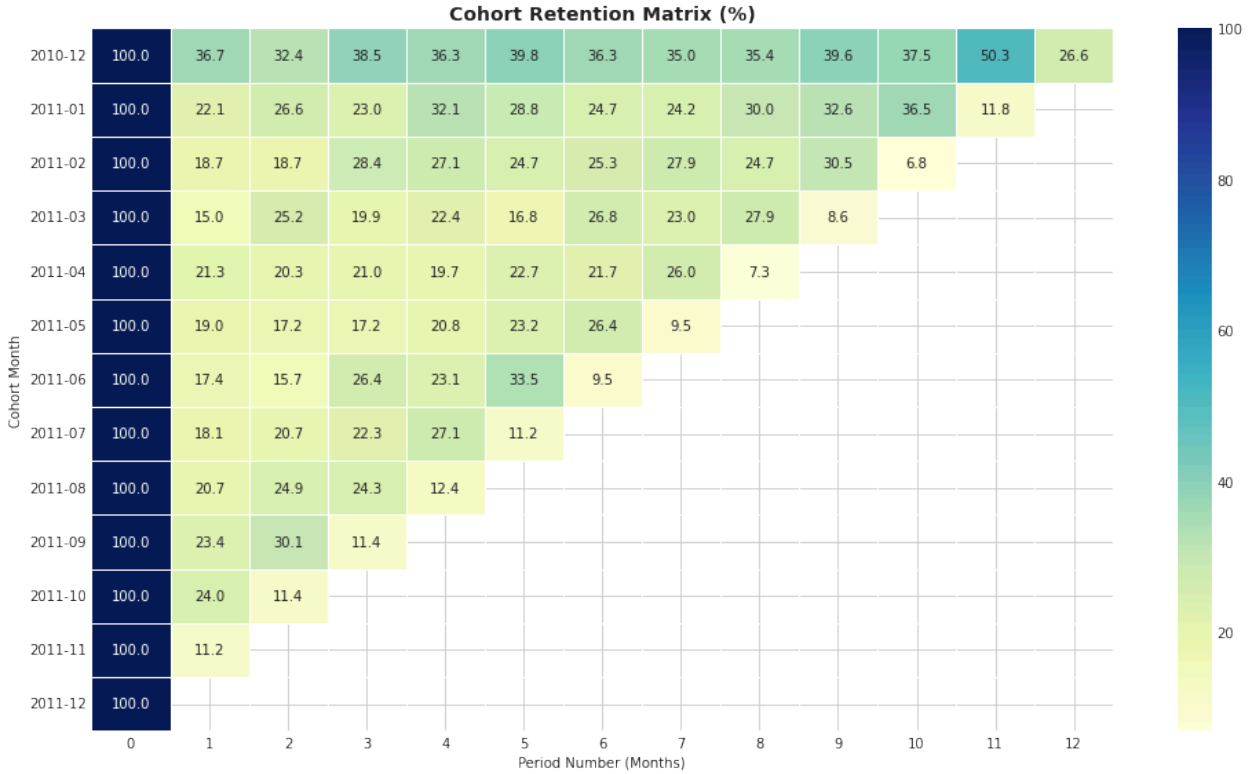


RFM saved to HDFS

In [20]:
```
print("Retention Analysis")

# first purchase month per customer (cohort)
customer_cohort = (df.groupBy("CustomerID")
                   .agg(F.min("YearMonth").alias("CohortMonth")))

df_cohort = df.join(customer_cohort, "CustomerID")

# Period number = months since first purchase
df_cohort = df_cohort.withColumn(
    "PeriodNumber",
    (F.year("YearMonth") * 12 + F.month("YearMonth")) -
    (F.year("CohortMonth") * 12 + F.month("CohortMonth"))
)

# Unique customers per cohort & period
cohort_data = (df_cohort.groupBy("CohortMonth", "PeriodNumber")
               .agg(F.countDistinct("CustomerID").alias("CustomerCount")))

# Cohort sizes
cohort_sizes = (customer_cohort.groupBy("CohortMonth")
                .agg(F.count("*").alias("CohortSize")))

cohort_table = cohort_data.join(cohort_sizes, "CohortMonth")
cohort_table = cohort_table.withColumn(
    "RetentionRate",
    F.round(F.col("CustomerCount") / F.col("CohortSize") * 100, 2)
)

# Pivot to matrix
```

```
cohort_matrix = (cohort_table
    .groupBy("CohortMonth")
    .pivot("PeriodNumber")
    .agg(F.first("RetentionRate"))
    .orderBy("CohortMonth")).toPandas()

# Plot heatmap
cohort_matrix_indexed = cohort_matrix.set_index("CohortMonth")
plt.figure(figsize=(14, 8))
sns.heatmap(cohort_matrix_indexed.iloc[:, :13],   # first 13 periods
            annot=True, fmt=".1f", cmap="YlGnBu", linewidths=0.5)
plt.title("Cohort Retention Matrix (%)", fontsize=14, fontweight="bold")
plt.ylabel("Cohort Month")
plt.xlabel("Period Number (Months)")
plt.tight_layout()
plt.savefig("/tmp/plot_08_cohort_retention.png", dpi=150)
plt.show()

cohort_table.write.mode("overwrite").parquet(f"{HDFS_OUTPUT}/cohort_retention"
print("saved to HDFS")
```

```
Retention Analysis
```



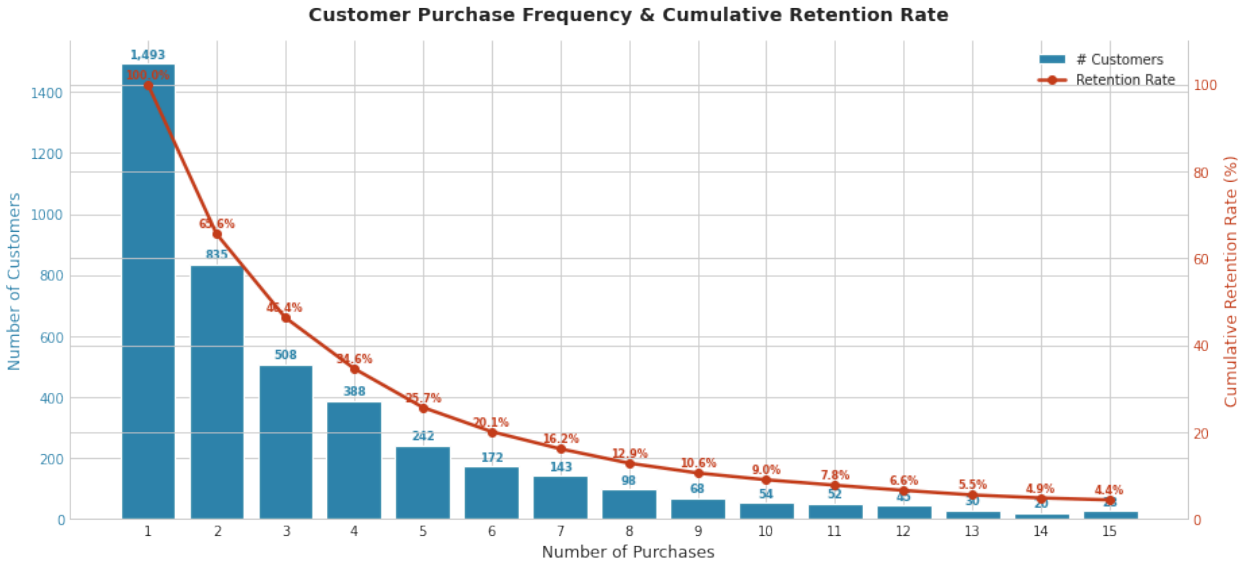

```
saved to HDFS
```

In [22]:
```
print("Retention Analysis")

# First purchase + total orders per customer
customer_stats = (df.groupBy("CustomerID")
    .agg(
```

```
        F.min("InvoiceDate").alias("FirstPurchase"),
        F.max("InvoiceDate").alias("LastPurchase"),
        F.countDistinct("InvoiceNo").alias("TotalOrders")
    ))

customer_stats = customer_stats.withColumn(
    "IsRetained", (F.col("TotalOrders") > 1).cast("int")
)

# Overall numbers
total    = customer_stats.count()
retained = customer_stats.filter(F.col("IsRetained") == 1).count()
one_time = total - retained
rate     = round(retained / total * 100, 2)

print(f"Total Customers : {total:,}")
print(f"Retained        : {retained:,}  ({rate}%)")
print(f"One-time only   : {one_time:,}  ({round(100 - rate, 2)}%)")

# Return frequency distribution (how many customers made exactly N purchases)
order_dist = (customer_stats
    .groupBy("TotalOrders")
    .agg(F.count("*").alias("CustomerCount"))
    .orderBy("TotalOrders")
    .toPandas())

order_dist["RetentionRate"] = (
    order_dist["CustomerCount"][::-1].cumsum()[::-1] / total * 100
).round(2)

# Plot
fig, ax1 = plt.subplots(figsize=(13, 6))

# Cap display at 15 orders (long tail is noise)
plot_df = order_dist[order_dist["TotalOrders"] <= 15].copy()

# Bar — number of customers per order count
ax1.bar(plot_df["TotalOrders"], plot_df["CustomerCount"],
        color="#2E86AB", edgecolor="white", linewidth=1.2, label="# Customers"

for _, row in plot_df.iterrows():
    ax1.text(row["TotalOrders"], row["CustomerCount"] + 10,
             f'{int(row["CustomerCount"]):,}',
             ha="center", va="bottom", fontsize=8.5, fontweight="bold", color=

ax1.set_xlabel("Number of Purchases", fontsize=12)
ax1.set_ylabel("Number of Customers", fontsize=12, color="#2E86AB")
ax1.tick_params(axis="y", labelcolor="#2E86AB")
ax1.spines["top"].set_visible(False)
ax1.set_xticks(plot_df["TotalOrders"])

# Line — cumulative retention rate (% of customers who made at least N purchas
ax2 = ax1.twinx()
```

```
ax2.plot(plot_df["TotalOrders"], plot_df["RetentionRate"],
         color="#C73E1D", linewidth=2.5, marker="o", markersize=6, label="Rete

for _, row in plot_df.iterrows():
    ax2.text(row["TotalOrders"], row["RetentionRate"] + 1.5,
             f'{row["RetentionRate"]:.1f}%',
             ha="center", fontsize=8, color="#C73E1D", fontweight="bold")

ax2.set_ylabel("Cumulative Retention Rate (%)", fontsize=12, color="#C73E1D")
ax2.tick_params(axis="y", labelcolor="#C73E1D")
ax2.spines["top"].set_visible(False)
ax2.set_ylim(0, 110)

# Legends
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc="upper right", fontsize=10)

plt.title("Customer Purchase Frequency & Cumulative Retention Rate",
          fontsize=14, fontweight="bold", pad=15)
plt.tight_layout()
plt.savefig("/tmp/plot_retention.png", dpi=150, bbox_inches="tight")
plt.show()
```

```
Retention Analysis
Total Customers : 4,339
Retained        : 2,846  (65.59%)
One-time only   : 1,493  (34.41%)
```



In [8]:
```
print("Product Portfolio Analysis")

# Revenue + quantity per product
product_stats = (df.groupBy("StockCode", "Description")
    .agg(
        F.round(F.sum("Revenue"), 2).alias("ProductRevenue"),
        F.sum("Quantity").alias("TotalQuantity"),
        F.countDistinct("InvoiceNo").alias("TotalOrders")
```

```
    ))

# Compute medians as split threshold
revenue_median  = product_stats.approxQuantile("ProductRevenue", [0.5], 0.01)[
quantity_median = product_stats.approxQuantile("TotalQuantity",  [0.5], 0.01)[

print(f"Revenue median  : £{revenue_median:,.2f}")
print(f"Quantity median : {quantity_median:,.0f} units")

# Classify into quadrants
product_stats = product_stats.withColumn(
    "Quadrant",
    F.when((F.col("ProductRevenue") >= revenue_median) & (F.col("TotalQuantity
     .when((F.col("ProductRevenue") >= revenue_median) & (F.col("TotalQuantity
     .when((F.col("ProductRevenue") <  revenue_median) & (F.col("TotalQuantity
     .otherwise("Low Revenue / Low Volume")
)

# Summary per quadrant
quadrant_summary = (product_stats.groupBy("Quadrant")
    .agg(
        F.count("*").alias("ProductCount"),
        F.round(F.sum("ProductRevenue"), 2).alias("TotalRevenue"),
        F.round(F.avg("ProductRevenue"), 2).alias("AvgRevenue"),
        F.round(F.avg("TotalQuantity"), 0).alias("AvgQuantity")
    )
    .orderBy(F.desc("TotalRevenue")))

print("\nQuadrant Summary:")
quadrant_summary.show(truncate=False)

# Top 5 per quadrant
print("\nTop 5 per Quadrant (by Revenue):")
for quadrant in ["High Revenue / High Volume", "High Revenue / Low Volume",
                 "Low Revenue / High Volume", "Low Revenue / Low Volume"]:
    print(f"\n-- {quadrant} --")
    (product_stats
        .filter(F.col("Quadrant") == quadrant)
        .orderBy(F.desc("ProductRevenue"))
        .select("StockCode", "Description", "ProductRevenue", "TotalQuantity",
        .limit(5)
        .show(truncate=False))

# --- Plot ---
plot_df  = product_stats.sample(fraction=0.8).limit(500).toPandas()
quad_pd  = quadrant_summary.toPandas()

colors = {
    "High Revenue / High Volume": "#2E86AB",
    "High Revenue / Low Volume":  "#A23B72",
    "Low Revenue / High Volume":  "#F18F01",
    "Low Revenue / Low Volume":   "#AAAAAA"
}
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Scatter — revenue vs quantity
for quadrant, group in plot_df.groupby("Quadrant"):
    axes[0].scatter(group["TotalQuantity"], group["ProductRevenue"],
                    label=quadrant, color=colors[quadrant],
                    alpha=0.7, s=50, edgecolors="white")
axes[0].axvline(quantity_median, color="gray", linestyle="--", linewidth=1)
axes[0].axhline(revenue_median,  color="gray", linestyle="--", linewidth=1)
axes[0].set_xlabel("Total Quantity Sold")
axes[0].set_ylabel("Total Revenue (£)")
axes[0].set_title("Product Portfolio Matrix", fontweight="bold", fontsize=13)
axes[0].legend(fontsize=8)
axes[0].spines["top"].set_visible(False)
axes[0].spines["right"].set_visible(False)

# Bar — product count per quadrant
bar_colors = [colors[q] for q in quad_pd["Quadrant"]]
bars = axes[1].bar(quad_pd["Quadrant"], quad_pd["ProductCount"],
                   color=bar_colors, edgecolor="white", linewidth=1.5)
for bar, rev in zip(bars, quad_pd["TotalRevenue"]):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5,
                 f'£{rev:,.0f}', ha="center", fontsize=9, fontweight="bold")
axes[1].set_title("Products per Quadrant\n(label = total revenue)", fontweight
axes[1].set_ylabel("Number of Products")
axes[1].tick_params(axis="x", labelsize=8)
axes[1].spines["top"].set_visible(False)
axes[1].spines["right"].set_visible(False)

plt.tight_layout()
plt.savefig("/tmp/plot_product_portfolio.png", dpi=150, bbox_inches="tight")
plt.show()

product_stats.write.mode("overwrite").parquet(f"{HDFS_OUTPUT}/product_portfoli
print("Saved to HDFS")
```

Product Portfolio Analysis
Revenue median : £607.73
Quantity median : 315 units

Quadrant Summary:

Quadrant	ProductCount	TotalRevenue	AvgRevenue	AvgQuantity
High Revenue / High Volume	1803	9829067.84	5451.51	2871.0
High Revenue / Low Volume	300	472911.15	1576.37	195.0
Low Revenue / Low Volume	1758	247995.44	141.07	64.0
Low Revenue / High Volume	300	116710.11	389.03	802.0

Top 5 per Quadrant (by Revenue):

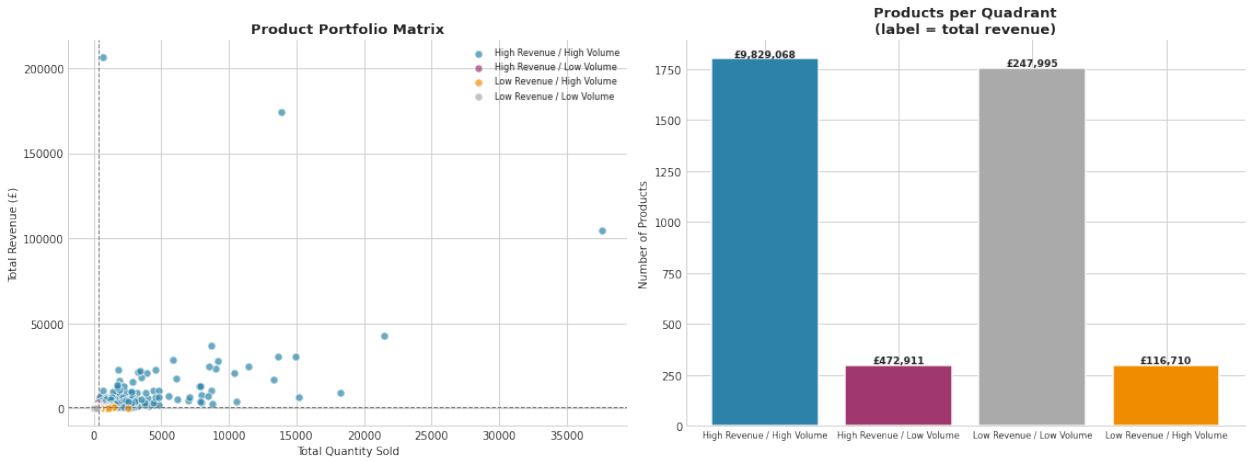
-- High Revenue / High Volume --

StockCode	Description	ProductRevenue	TotalQuantity	TotalOrders
DOT	DOTCOM POSTAGE	206248.77	706	706
22423	REGENCY CAKESTAND 3 TIER	174484.74	13879	1988
23843	PAPER CRAFT , LITTLE BIRDIE	168469.6	80995	1
85123A	WHITE HANGING HEART T-LIGHT HOLDER	104340.29	37599	2189
47566	PARTY BUNTING	99504.33	18295	1685

-- High Revenue / Low Volume --

```
+---------+-----------------------------+-------------+------------
+-----------+
|StockCode|Description                  |ProductRevenue|TotalQuantity|TotalOrd
ers|
+---------+-----------------------------+-------------+------------
+-----------+
|22502    |PICNIC BASKET WICKER 60 PIECES|39619.5       |61           |2
|
|AMAZONFEE|AMAZON FEE                   |13761.09      |2            |2
|
|B        |Adjust bad debt              |11062.06      |1            |1
|
|22655    |VINTAGE RED KITCHEN CABINET  |8125.0        |60           |38
|
|C2       |CARRIAGE                     |7051.0        |142          |141
|
+---------+-----------------------------+-------------+------------
+-----------+


-- Low Revenue / High Volume --
+---------+---------------------------------+-------------+------------
+-----------+
|StockCode|Description                      |ProductRevenue|TotalQuantity|Tota
lOrders|
+---------+---------------------------------+-------------+------------
+-----------+
|72598    |SET/12 TAPER CANDLES             |605.89        |607          |93
|
|22595    |CHRISTMAS GINGHAM HEART          |605.8         |649          |54
|
|22999    |TRAVEL CARD WALLET VINTAGE LEAF  |605.78        |1415         |117
|
|84625A   |PINK NEW BAROQUECANDLESTICK CANDLE|605.48        |478          |42
|
|22232    |JIGSAW TOADSTOOLS 3 PIECE        |604.46        |690          |83
|
+---------+---------------------------------+-------------+------------
+-----------+


-- Low Revenue / Low Volume --
+---------+---------------------------------+-------------+------------
+-----------+
|StockCode|Description                       |ProductRevenue|TotalQuantity|Tot
alOrders|
+---------+---------------------------------+-------------+------------
+-----------+
|21769    |VINTAGE POST OFFICE CABINET       |607.65        |11           |2
|
|21709    |FOLDING UMBRELLA CHOCOLATE POLKADOT|607.23        |133          |59
|
|23520    |EMBROIDERED RIBBON REEL RUBY      |604.3         |168          |32
|
```

```
|22642    |SET OF 4 NAPKIN CHARMS STARS       |596.36        |269          |51
|
|21253    |SET OF PICTURE FRAME  STICKERS     |593.37        |135          |62
|
+---------+-----------------------------------+--------------+-------------
+-----------+
```




```
[Stage 222:========================>                                 (3 + 4) /
7]
Saved to HDFS
```

ML Engineer

In [19]:
```python
import findspark
findspark.init()


from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import *
from pyspark.sql.window import Window
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression, RandomForestRegressor, GBT
from pyspark.ml.evaluation import RegressionEvaluator
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

spark = SparkSession.builder \
    .appName("ML_DemandForecasting") \
    .master("local[*]") \
    .getOrCreate()

HDFS_CLEANED = "hdfs://localhost:9000/data/retail/cleaned"
HDFS_OUTPUT  = "hdfs://localhost:9000/data/retail/outputs"
HDFS_MODELS  = "hdfs://localhost:9000/data/retail/models"
```

In [20]:
```
# agreggate weekly demand
df = spark.read.parquet(HDFS_CLEANED)

weekly = (df.groupBy("Year", "WeekOfYear", "StockCode")
           .agg(
               F.sum("Quantity").alias("Demand"),
               F.sum("Revenue").alias("Revenue"),
               F.countDistinct("InvoiceNo").alias("NumOrders"),
               F.countDistinct("CustomerID").alias("NumCustomers"),
               F.avg("UnitPrice").alias("AvgPrice")
           )
           .orderBy("StockCode", "Year", "WeekOfYear"))

print(f"rows: {weekly.count()}")
weekly.show(5)
```

```
rows: 101579
[Stage 722:>                                                        (0 + 8) /
8]
+----+----------+---------+------+-----------------+---------+------------
+------------------+
|Year|WeekOfYear|StockCode|Demand|          Revenue|NumOrders|NumCustomers|
AvgPrice|
+----+----------+---------+------+-----------------+---------+------------
+------------------+
|2010|        48|    10002|    70|65.16999999999999|        7|           5|1.08
14285714285714|
|2010|        49|    10002|   138|           127.02|       14|           9| 1.1
97142857142857|
|2010|        50|    10002|    41|38.89999999999999|        8|           5|
1.255|
|2010|        51|    10002|     2|             3.32|        1|           1|
1.66|
|2011|         1|    10002|    73|            62.86|        3|           3|1.11
99999999999999|
+----+----------+---------+------+-----------------+---------+------------
+------------------+
only showing top 5 rows
```

In [21]:
```
print("Feature Engineering")

# Fix types at the root
weekly = (weekly
    .withColumn("Demand",      F.col("Demand").cast("double"))
    .withColumn("AvgPrice",    F.col("AvgPrice").cast("double"))
    .withColumn("WeekOfYear",  F.col("WeekOfYear").cast("double"))
    .withColumn("Year",        F.col("Year").cast("double"))
    .withColumn("NumOrders",   F.col("NumOrders").cast("double"))
    .withColumn("NumCustomers",F.col("NumCustomers").cast("double"))
)
```

```
product_window = Window.partitionBy("StockCode").orderBy("Year", "WeekOfYear")

# Lag features
weekly = (weekly
    .withColumn("Lag1", F.lag("Demand", 1).over(product_window))
    .withColumn("Lag2", F.lag("Demand", 2).over(product_window))
    .withColumn("Lag4", F.lag("Demand", 4).over(product_window))
    .withColumn("Lag8", F.lag("Demand", 8).over(product_window))
)

# Rolling statistics
weekly = (weekly
    .withColumn("RollingMean4", F.avg("Demand").over(product_window.rowsBetwee
    .withColumn("RollingMean8", F.avg("Demand").over(product_window.rowsBetwee
    .withColumn("RollingStd4",  F.stddev("Demand").over(product_window.rowsBet
)

# Seasonality
weekly = (weekly
    .withColumn("WeekSin", F.sin(2 * np.pi * F.col("WeekOfYear") / 52))
    .withColumn("WeekCos", F.cos(2 * np.pi * F.col("WeekOfYear") / 52))
)

# Price features
weekly = weekly.withColumn("PriceLag1", F.lag("AvgPrice", 1).over(product_winc

# Drop nulls from lags
weekly = weekly.dropna()
weekly = weekly.withColumn("Demand", F.col("Demand").cast("double"))  # ensure

print(f"Rows after feature eng: {weekly.count()}")
weekly.printSchema()  # verify all doubles
```

```
[Stage 727:================================================>    (183 + 9) / 20
0]
```

Feature Engineering

```
[Stage 769:============> (181 + 8) / 200][Stage 776:>                (0 + 0) /
1]
```

```
Rows after feature eng: 73723
root
 |-- Year: double (nullable = true)
 |-- WeekOfYear: double (nullable = true)
 |-- StockCode: string (nullable = true)
 |-- Demand: double (nullable = true)
 |-- Revenue: double (nullable = true)
 |-- NumOrders: double (nullable = false)
 |-- NumCustomers: double (nullable = false)
 |-- AvgPrice: double (nullable = true)
 |-- Lag1: double (nullable = true)
 |-- Lag2: double (nullable = true)
 |-- Lag4: double (nullable = true)
 |-- Lag8: double (nullable = true)
 |-- RollingMean4: double (nullable = true)
 |-- RollingMean8: double (nullable = true)
 |-- RollingStd4: double (nullable = true)
 |-- WeekSin: double (nullable = true)
 |-- WeekCos: double (nullable = true)
 |-- PriceLag1: double (nullable = true)
```

In [22]:
```
# split time based
# Train: Dec 2010 → Sep 2011
# Test:  Oct 2011 → Dec 2011

# clac expanding mean along

train_df = weekly.filter(
    (F.col("Year") == 2010) |
    ((F.col("Year") == 2011) & (F.col("WeekOfYear") <= 39))
).withColumn("ExpandingMean",
    F.avg("Demand").over(product_window.rowsBetween(Window.unboundedPreceding,

# last known ExpandingMean from train per product
last_expanding = (train_df
    .groupBy("StockCode")
    .agg(F.last("ExpandingMean", ignorenulls=True).alias("ExpandingMean")))

test_df = (weekly
    .filter((F.col("Year") == 2011) & (F.col("WeekOfYear") > 39))
    .drop("ExpandingMean")
    .join(last_expanding, "StockCode", "left"))
print(train_df.count(), test_df.count())
```

```
[Stage 853:==========>  (154 + 8) / 200][Stage 860:>                 (0 + 0) /
1]
52931 20792
```

In [23]:
```
# historical product features
product_profile = (train_df.groupBy("StockCode")
```

```
    .agg(
        F.avg("Demand").alias("ProductAvgDemand"),
        F.stddev("Demand").alias("ProductDemandStd"),
        ((F.last("Demand") - F.first("Demand")) / F.count("*")).alias("Product
    )
    .fillna(0))

train_df = train_df.join(product_profile, "StockCode", "left")
test_df  = test_df.join(product_profile, "StockCode", "left")

# # Encode StockCode
# from pyspark.ml.feature import StringIndexer
# indexer_model = StringIndexer(inputCol="StockCode", outputCol="StockCodeIdx"
# train_df = indexer_model.transform(train_df)
# test_df = indexer_model.transform(test_df)
```

In [24]:
```
feature_cols = [
#     "StockCodeIdx",
    "ProductAvgDemand", "ProductDemandStd", "ProductTrend",
    "WeekOfYear", "Year", "WeekSin", "WeekCos",
    "NumOrders", "NumCustomers", "AvgPrice", "PriceLag1",
    "Lag1", "Lag2", "Lag4", "Lag8",
    "RollingMean4", "RollingMean8", "RollingStd4",
    "ExpandingMean"
]

assembler = VectorAssembler(inputCols=feature_cols, outputCol="features", hand
train_vec = assembler.transform(train_df).select("features", F.col("Demand").a
test_vec  = assembler.transform(test_df).select("features", F.col("Demand").al

train_vec.cache()
test_vec.cache()
```

```
[Stage 865:=====================================================>(197 + 3) / 20
0]
```

Out[24]:
```
DataFrame[features: vector, label: double, StockCode: string, Year: double, W
eekOfYear: double]
```

In [25]:
```
def evaluate(df_pred, name):
    evaluator = RegressionEvaluator(labelCol="label", predictionCol="predictic
    rmse = evaluator.setMetricName("rmse").evaluate(df_pred)
    mae  = evaluator.setMetricName("mae").evaluate(df_pred)
    r2   = evaluator.setMetricName("r2").evaluate(df_pred)
    mape = (df_pred
        .filter(F.col("label") > 0)
        .withColumn("pct", F.abs((F.col("label") - F.col("prediction")) / F.cc
        .agg(F.avg("pct")).collect()[0][0]) * 100
    print(f"{name:<30} RMSE:{round(rmse,2):>8}  MAE:{round(mae,2):>8}  MAPE:{r
    return {"RMSE": round(rmse,2), "MAE": round(mae,2), "MAPE": round(mape,2),

results = {}

# baselines
```

```
print("Baselines")
results["Naive (Lag1)"] = evaluate(
    test_df.withColumn("prediction", F.col("Lag1"))
           .select(F.col("Demand").alias("label"), "prediction"), "Naive (Lag1

results["MA-4"] = evaluate(
    test_df.withColumn("prediction", F.col("RollingMean4"))
           .select(F.col("Demand").alias("label"), "prediction"), "MA-4")

print("Linear Regression")
from pyspark.ml.regression import LinearRegression
lr = LinearRegression(featuresCol="features", labelCol="label",
                      maxIter=100, regParam=0.1, elasticNetParam=0.0)
lr_model  = lr.fit(train_vec)
lr_preds  = lr_model.transform(test_vec)
results["Linear Regression"] = evaluate(lr_preds, "Linear Regression")

print("Random Forest")
from pyspark.ml.regression import RandomForestRegressor
rf = RandomForestRegressor(featuresCol="features", labelCol="label",
                           numTrees=20, maxDepth=4, seed=42, maxBins=3000)
rf_model  = rf.fit(train_vec)
rf_preds  = rf_model.transform(test_vec)
results["Random Forest"] = evaluate(rf_preds, "Random Forest")

print("Gradient Boosting")
from pyspark.ml.regression import GBTRegressor
gbt = GBTRegressor(featuresCol="features", labelCol="label",
                   maxIter=20, maxDepth=4, stepSize=0.1, seed=42, maxBins=3000
gbt_model = gbt.fit(train_vec)
gbt_preds = gbt_model.transform(test_vec)
results["Gradient Boosting"] = evaluate(gbt_preds, "Gradient Boosting")
```

```
Baselines

Naive (Lag1)                  RMSE:  144.07  MAE:   50.34  MAPE:204.81%  R2:
0.2951

MA-4                          RMSE:  121.47  MAE:   44.76  MAPE:205.98%  R2:
0.4989
Linear Regression

Linear Regression             RMSE:  131.81  MAE:   49.44  MAPE:313.76%  R2:
0.4409
Random Forest

Random Forest                 RMSE:  137.93  MAE:   43.81  MAPE:229.02%  R2:
0.3878
Gradient Boosting

Gradient Boosting             RMSE:  126.72  MAE:   41.48  MAPE:156.16%  R2:
0.4833
```

In [26]:

```
print(f"{'Model':<30} {'RMSE':>8} {'MAE':>8} {'MAPE':>8} {'R2':>8}")
for model, m in results.items():
    r2 = str(m["R2"]) if m["R2"] is not None else "N/A"
    print(f"{model:<30} {str(m['RMSE']):>8} {str(m['MAE']):>8} {str(m['MAPE'])

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Get predictions from all models
models_to_plot = {
    "Linear Regression": lr_preds,
    "Random Forest": rf_preds,
    "Gradient Boosting": gbt_preds
}

for idx, (model_name, predictions) in enumerate(models_to_plot.items()):
    row, col = divmod(idx, 2)

    # Sample 200 points for visualization
    plot_df = predictions.select("label", "prediction").toPandas().head(200)

    axes[row, col].plot(plot_df["label"].values, label="Actual", color="#2E86A
    axes[row, col].plot(plot_df["prediction"].values, label=f"Predicted", colo
    axes[row, col].set_title(f"{model_name}\n(RMSE: {results[model_name]['RMSE
                             fontweight="bold", fontsize=12)
    axes[row, col].set_xlabel("Sample Index")
    axes[row, col].set_ylabel("Weekly Demand")
    axes[row, col].legend()
    axes[row, col].spines["top"].set_visible(False)
    axes[row, col].spines["right"].set_visible(False)

plt.suptitle("Actual vs Predicted Demand - Model Comparison", fontweight="bold
plt.tight_layout()

# residual analysis
fig2, axes2 = plt.subplots(1, 3, figsize=(15, 5))

for idx, (model_name, predictions) in enumerate(models_to_plot.items()):
    # Calculate residuals
    resid_df = predictions.select(
        "label", "prediction",
        (F.col("label") - F.col("prediction")).alias("residual")
    ).toPandas()

    # Plot residuals
    axes2[idx].scatter(resid_df["prediction"], resid_df["residual"], alpha=0.5
    axes2[idx].axhline(y=0, color='red', linestyle='--', linewidth=1)
    axes2[idx].set_title(f"{model_name}", fontweight="bold", fontsize=12)
    axes2[idx].set_xlabel("Predicted Demand")
    axes2[idx].set_ylabel("Residuals (Actual - Predicted)")
    axes2[idx].spines["top"].set_visible(False)
    axes2[idx].spines["right"].set_visible(False)

plt.suptitle("Residual Plot Analysis", fontweight="bold", fontsize=14)
```

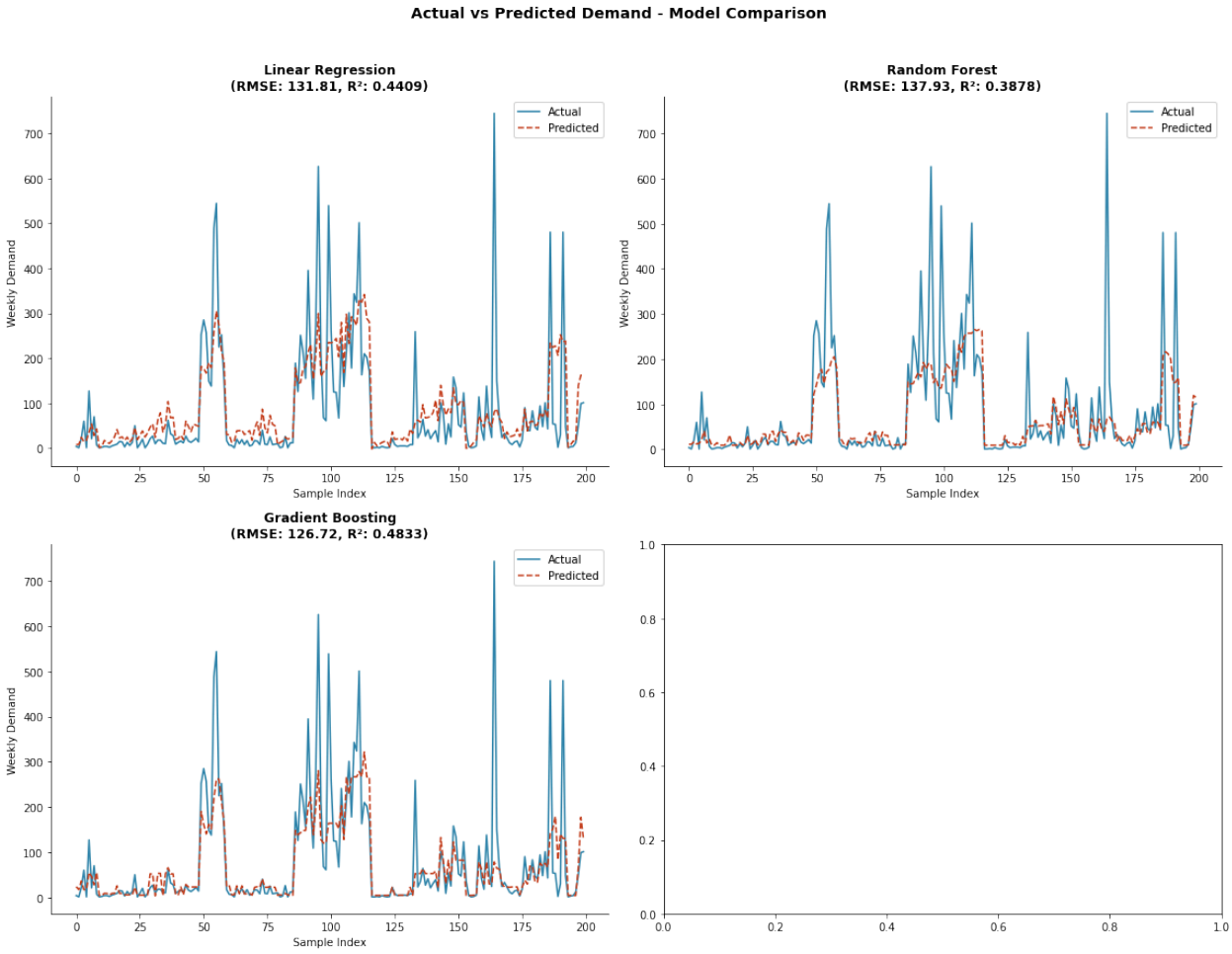
```
plt.tight_layout()

# feature importance
fig3, axes3 = plt.subplots(1, 2, figsize=(14, 8))

# Random Forest Feature Importance
rf_importances = rf_model.featureImportances.toArray()
rf_feat_imp = (pd.DataFrame({"Feature": feature_cols, "Importance": rf_importa
               .sort_values("Importance", ascending=True)
               .tail(15))

axes3[0].barh(rf_feat_imp["Feature"], rf_feat_imp["Importance"], color="#2E86A
axes3[0].set_title("Random Forest - Feature Importance (Top 15)", fontweight="
axes3[0].set_xlabel("Importance")
axes3[0].spines["top"].set_visible(False)
axes3[0].spines["right"].set_visible(False)

# Gradient Boosting Feature Importance
gbt_importances = gbt_model.featureImportances.toArray()
gbt_feat_imp = (pd.DataFrame({"Feature": feature_cols, "Importance": gbt_impor
                .sort_values("Importance", ascending=True)
                .tail(15))

axes3[1].barh(gbt_feat_imp["Feature"], gbt_feat_imp["Importance"], color="#A23
axes3[1].set_title("Gradient Boosting - Feature Importance (Top 15)", fontweig
axes3[1].set_xlabel("Importance")
axes3[1].spines["top"].set_visible(False)
axes3[1].spines["right"].set_visible(False)

plt.suptitle("Feature Importance Comparison", fontweight="bold", fontsize=14)
plt.tight_layout()

fig4, ax4 = plt.subplots(figsize=(10, 6))

error_data = []
model_names = []
for model_name, predictions in models_to_plot.items():
    errors = predictions.select(
        (F.abs(F.col("label") - F.col("prediction"))).alias("abs_error")
    ).toPandas()["abs_error"].values
    error_data.append(errors)
    model_names.append(model_name)

bp = ax4.boxplot(error_data, labels=model_names, patch_artist=True)
for patch, color in zip(bp['boxes'], ['#2E86AB', '#A23B72', '#C73E1D']):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)

ax4.set_title("Absolute Error Distribution by Model", fontweight="bold", fonts
ax4.set_ylabel("Absolute Error")
ax4.set_yscale('log')  # Log scale if errors have wide range
ax4.spines["top"].set_visible(False)
ax4.spines["right"].set_visible(False)
```

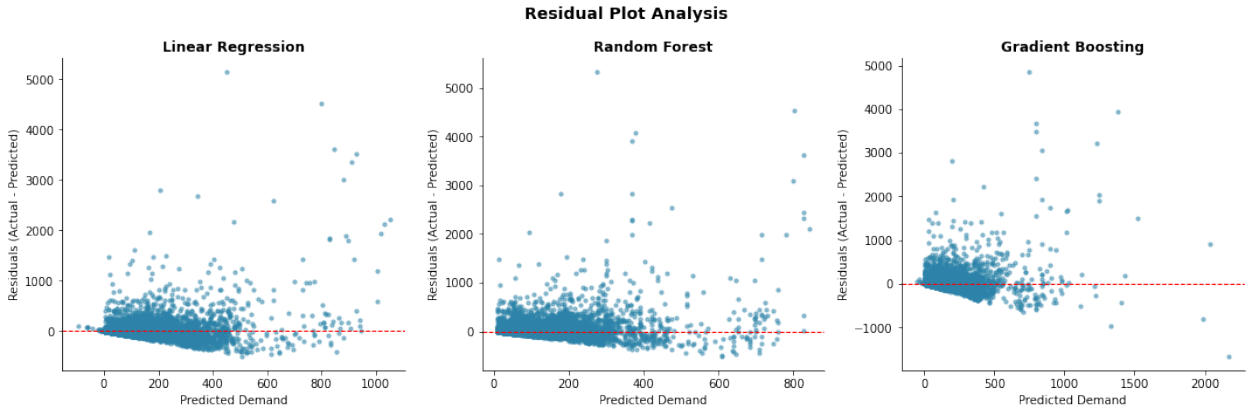
```
ax4.grid(axis='y', alpha=0.3)

plt.tight_layout()

fig5, axes5 = plt.subplots(1, 3, figsize=(15, 5))

for idx, (model_name, predictions) in enumerate(models_to_plot.items()):
    plot_df = predictions.select("label", "prediction").toPandas()

    axes5[idx].scatter(plot_df["label"], plot_df["prediction"], alpha=0.3, s=5

    # Perfect prediction line
    max_val = max(plot_df["label"].max(), plot_df["prediction"].max())
    axes5[idx].plot([0, max_val], [0, max_val], 'r--', alpha=0.5, label="Perfe

    axes5[idx].set_title(f"{model_name}\nR² = {results[model_name]['R2']}", fo
    axes5[idx].set_xlabel("Actual Demand")
    axes5[idx].set_ylabel("Predicted Demand")
    axes5[idx].set_xlim(0, max_val)
    axes5[idx].set_ylim(0, max_val)
    axes5[idx].legend()
    axes5[idx].spines["top"].set_visible(False)
    axes5[idx].spines["right"].set_visible(False)

plt.suptitle("Actual vs Predicted - Scatter Plot Analysis", fontweight="bold",
plt.tight_layout()

try:
    lr_model.write().overwrite().save(f"{HDFS_MODELS}/lr_model")
except NameError:
    print("Linear Regression model not available for saving")


summary = pd.DataFrame(results).T
summary.to_csv("/tmp/model_summary.csv")


print("Summary:")
print(summary.round(2).to_string())
```

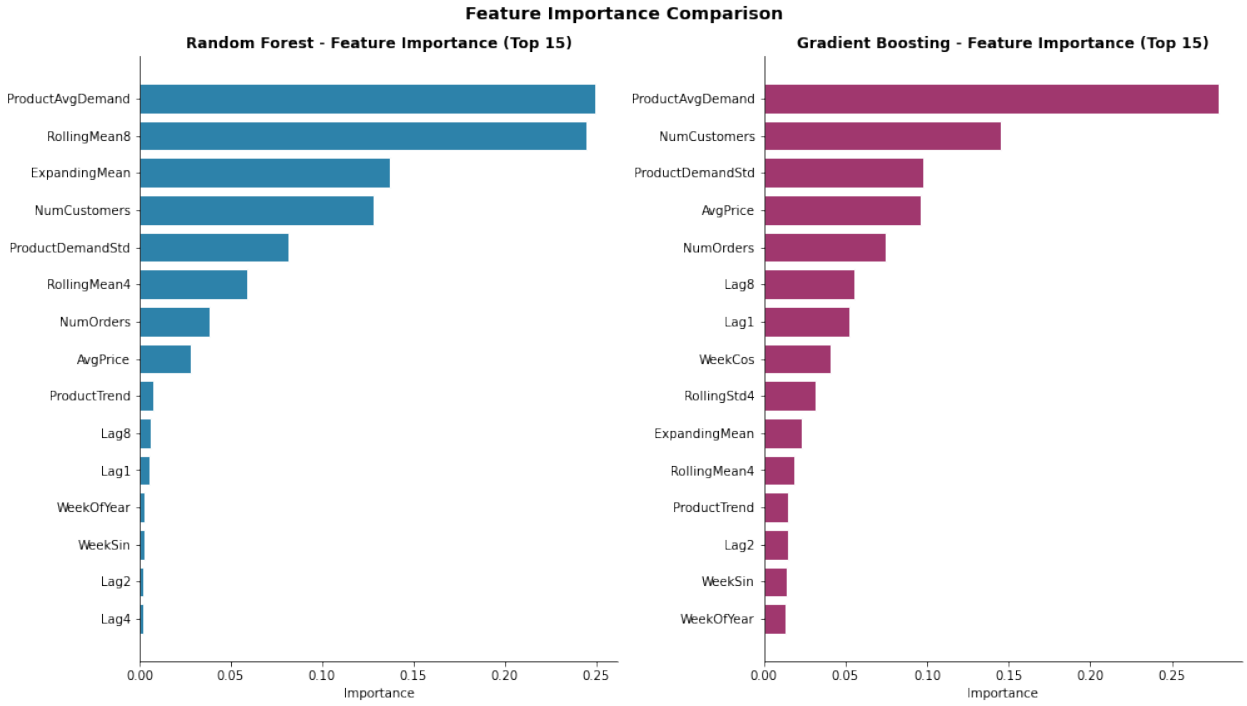
```
Model                              RMSE      MAE     MAPE       R2
Naive (Lag1)                     144.07    50.34  204.81%   0.2951
MA-4                             121.47    44.76  205.98%   0.4989
Linear Regression                131.81    49.44  313.76%   0.4409
Random Forest                    137.93    43.81  229.02%   0.3878
Gradient Boosting                126.72    41.48  156.16%   0.4833
/tmp/ipykernel_10629/4035172233.py:97: MatplotlibDeprecationWarning: The 'label
s' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9;
support for the old name will be dropped in 3.11.
  bp = ax4.boxplot(error_data, labels=model_names, patch_artist=True)
```

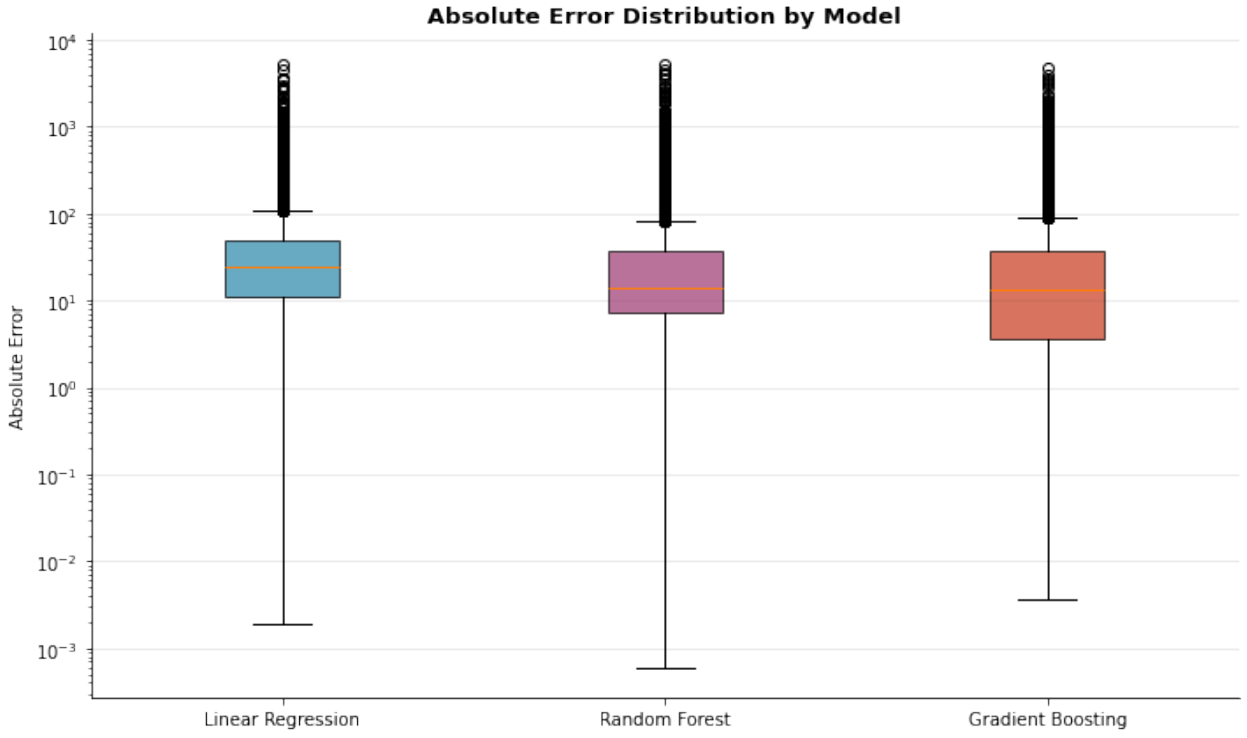
Summary:

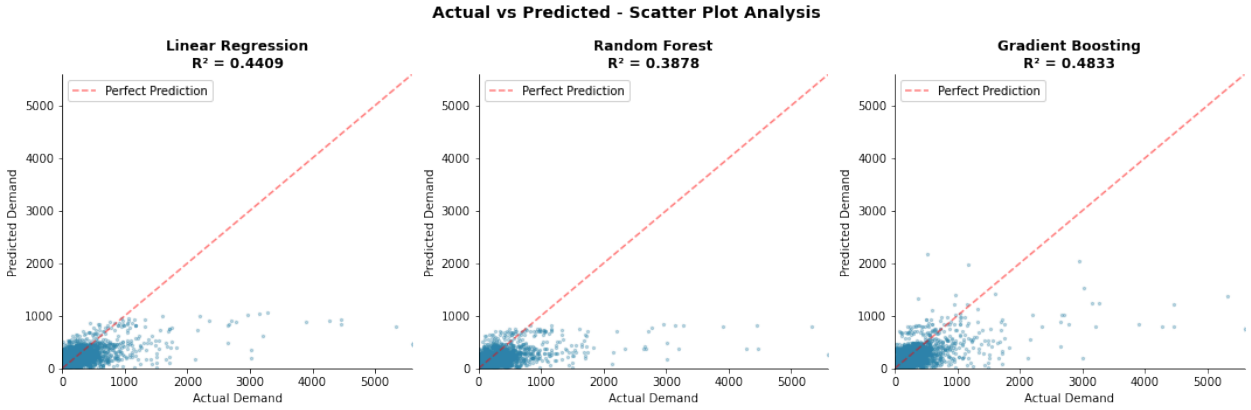
	RMSE	MAE	MAPE	R2
Naive (Lag1)	144.07	50.34	204.81	0.30
MA-4	121.47	44.76	205.98	0.50
Linear Regression	131.81	49.44	313.76	0.44
Random Forest	137.93	43.81	229.02	0.39
Gradient Boosting	126.72	41.48	156.16	0.48






Feature Importance Comparison
Random Forest - Feature Importance (Top 15)
ProductAvgDemand
RollingMean8
ExpandingMean
NumCustomers
ProductDemandStd
RollingMean4
NumOrders
AvgPrice
ProductTrend
Lag8
Lag1
WeekOfYear
WeekSin
Lag2
Lag4
0.00
0.05
0.10
0.15
0.20
0.25
Importance
Gradient Boosting - Feature Importance (Top 15)
ProductAvgDemand
NumCustomers
ProductDemandStd
AvgPrice
NumOrders
Lag8
Lag1
WeekCos
RollingStd4
ExpandingMean
RollingMean4
ProductTrend
Lag2
WeekSin
WeekOfYear
0.00
0.05
0.10
0.15
0.20
0.25
Importance


Absolute Error Distribution by Model
Absolute Error
10^{4}
10^{3}
10^{2}
10^{1}
10^{0}
10^{-1}
10^{-2}
10^{-3}
Linear Regression
Random Forest
Gradient Boosting



In [27]:
```
spark.stop()
```

Big Data Eng

In [28]:
```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
import time

spark = SparkSession.builder \
    .appName("BD_DemandForecasting") \
    .master("local[*]") \
    .getOrCreate()


HDFS_CLEANED = "hdfs://localhost:9000/data/retail/cleaned"

df = spark.read.parquet(HDFS_CLEANED)
print(f"Loaded {df.count()} rows")
```

```
[Stage 1:================>                                        (2 + 5) /
7]
Loaded 530104 rows
```

In [29]:
```
print(f"Partitions in raw df: {df.rdd.getNumPartitions()}")

# Repartition for product-level aggregations
df_repart = df.repartition(8, "StockCode")
print(f"Partitions after repartition(8, StockCode): {df_repart.rdd.getNumParti
```

```
Partitions in raw df: 7
[Stage 4:=================================================>       (6 + 1) /
7]
Partitions after repartition(8, StockCode): 8
```

In [30]:
```
# Without repartition
start = time.time()
result1 = df.groupBy("StockCode").agg(F.sum("Quantity").alias("TotalQty")).col
```

```
t1 = time.time() - start
print(f"Without repartition: {t1:.2f}s")

# With repartition
start = time.time()
result2 = df_repart.groupBy("StockCode").agg(F.sum("Quantity").alias("TotalQty
t2 = time.time() - start
print(f"With repartition:    {t2:.2f}s")
print(f"Speedup: {t1/t2:.2f}x")
```

```
Without repartition: 2.29s
```

```
[Stage 8:=========================>                                 (3 + 4) /
7]
```

```
With repartition:    1.85s
Speedup: 1.23x
```

In [31]:
```
# Before cache
start = time.time()
df.filter(F.col("Country") == "United Kingdom").count()
df.filter(F.col("Country") == "United Kingdom").count()
t_no_cache = time.time() - start
print(f"Without cache : {t_no_cache:.2f}s")

# After cache
df_cached = df.cache()
df_cached.count()

start = time.time()
df_cached.filter(F.col("Country") == "United Kingdom").count()
df_cached.filter(F.col("Country") == "United Kingdom").count()
t_cache = time.time() - start
print(f"With cache:    {t_cache:.2f}s")
print(f"Cache speedup: {t_no_cache/t_cache:.2f}x")
```

```
Without cache : 3.20s
```

```
With cache:    0.82s
Cache speedup: 3.89x
```

In [32]:
```
print(f"Partitions before coalesce: {df.rdd.getNumPartitions()}")
df_coalesced = df.coalesce(4)
print(f"Partitions after coalesce(4): {df_coalesced.rdd.getNumPartitions()}")

# Write optimized output
HDFS_OPTIMIZED = "hdfs://localhost:9000/data/retail/optimized"
df_coalesced.write.mode("overwrite").parquet(HDFS_OPTIMIZED)
print(f"Coalesced output written to {HDFS_OPTIMIZED}")
```

```
Partitions before coalesce: 7
Partitions after coalesce(4): 4
```

```
Coalesced output written to hdfs://localhost:9000/data/retail/optimized
```

In [33]:
```
print("Execution Plan (groupBy StockCode)")
df.groupBy("StockCode").agg(F.sum("Quantity").alias("TotalQty")).explain()
```

```
Execution Plan (groupBy StockCode)
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[StockCode#38492], functions=[sum(Quantity#38494)])
   +- Exchange hashpartitioning(StockCode#38492, 200), ENSURE_REQUIREMENTS, [pl
an_id=33478]
      +- HashAggregate(keys=[StockCode#38492], functions=[partial_sum(Quantit
y#38494)])
         +- InMemoryTableScan [StockCode#38492, Quantity#38494]
               +- InMemoryRelation [InvoiceNo#38491, StockCode#38492, Descripti
on#38493, Quantity#38494, InvoiceDate#38495, UnitPrice#38496, CustomerID#38497,
Country#38498, IsCancelled#38499, Revenue#38500, Year#38501, Month#38502, Day#3
8503, DayOfWeek#38504, WeekOfYear#38505, Quarter#38506, Hour#38507, IsWeekend#3
8508, YearMonth#38509], StorageLevel(disk, memory, deserialized, 1 replicas)
                     +- *(1) ColumnarToRow
                        +- FileScan parquet [InvoiceNo#38491,StockCode#38492,De
scription#38493,Quantity#38494,InvoiceDate#38495,UnitPrice#38496,CustomerID#384
97,Country#38498,IsCancelled#38499,Revenue#38500,Year#38501,Month#38502,Day#385
03,DayOfWeek#38504,WeekOfYear#38505,Quarter#38506,Hour#38507,IsWeekend#38508,Ye
arMonth#38509] Batched: true, DataFilters: [], Format: Parquet, Location: InMem
oryFileIndex(1 paths)[hdfs://localhost:9000/data/retail/cleaned], PartitionFilt
ers: [], PushedFilters: [], ReadSchema: struct<InvoiceNo:string,StockCode:strin
g,Description:string,Quantity:int,InvoiceDate:timestamp,Un...
```

In [37]:
```
import subprocess
# replication on hdfs
subprocess.run(["hdfs", "dfs", "-stat", "%r",
    "/data/retail/cleaned/YearMonth=2010-12/part-00000-4badb10d-eb1b-4871-a5f2
```

```
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further detai
ls.
3
```

Out[37]:
```
CompletedProcess(args=['hdfs', 'dfs', '-stat', '%r', '/data/retail/cleaned/Ye
arMonth=2010-12/part-00000-4badb10d-eb1b-4871-a5f2-e6f0093826cd.c000.snappy.p
arquet'], returncode=0)
```

In [ ]:
```
```